# A superscalar Out of Order design for the RISC-V CVA6 Core

Telmo Fieux
*Université Toulouse III – Paul Sabatier*
Toulouse, France
fieuxtelmo@gmail.com

***Abstract*—This small article presents an Out of Order implementation for the RISC-V open source processor named CVA6. First we will go over the specifics of this processor, then we will explain the details of the micro-architecture developed. Finally, we will discuss our results: compared to the dual-issue in-order core, +32 % IPC at equal frequency, +5 % performance with each core at its own Fmax, and 1.62× fewer cycles on independent cache misses. And, analyze them as well as talk about the limits and future possible improvements of this design.**

***Index Terms*—Micro-architecture, Chip design, Out of Order execution, CVA6**

## I. Introduction

The CVA6 is an open source RISC-V processor originally developed by the University of Bologna and ETH Zurich. Later, the OpenHW Group was created as a non-profit organization to continue the development and make it a usable industrial product. As a final project for my master's degree I developed an Out of Order version of this processor that I will present in this article.

## II. The base architecture

The CVA6 is a 6-stage pipelined CPU. These stages are: PC, Fetch, Decode, Issue, Execute, Commit. It is highly customizable and thus supports a lot of features. To name a few: branch prediction, multiple levels of privilege, memory virtualization, memory management unit… It has different types of cache, the one used in our configuration being the HPD cache. It supports 32-bit mode (the one used in this architecture) as well as 64-bit mode. It also implements various RISC-V extensions such as floating-point instructions or compressed instructions, to mention a few. And finally it supports a superscalar mode which only supports dual issue, adds a new ALU module and will be the configuration that we will be interested in. In the next section we will explain in detail the base architecture of some parts of the CVA6 that have been heavily modified to implement the Out of Order (OoO) execution. Those parts are mainly: the issue stage, the execute stage and the load store unit.

### A. Issue Stage

The main purpose of this stage is to be the center of the CPU. It tracks the progress of each decoded instruction until its commit. It performs forwarding, access to the registers and commit in order. In the base design, the issue stage is composed of 2 modules: a **Scoreboard** (SB) and **Issue_Read_Operands** (IRO). The Scoreboard's main purpose is to resolve dependencies and ensure in-order commit, since the write back can be done out of order. Thus it holds a lot of status information for each instruction. It is basically a circular buffer with one entry for each instruction between the issue stage and the commit stage. For each instruction it contains various important data such as exception state, result, operands to verify dependencies, branch prediction validity etc. Plus, in combination with IRO, it helps track the state of each instruction through the pipeline to perform data forwarding. This structure will be the foundation of our Reorder Buffer. As for IRO, it accesses the registers, builds the instruction payload that will be sent next cycle to the execute stage, checks the availability of each functional unit and, finally, also has a big portion dedicated to tracking the state of each instruction through the pipeline to perform forwarding. We will not go over the forwarding and bypass mechanism since they have been entirely removed from the OoO design and, therefore, are irrelevant to this article.

### B. Execution Stage

The CVA6, being a modular CPU, has multiple functional units (FU) that will be disabled or not depending on the configuration. First, it has a maximum of 5 write back ports. The first one is shared between all the fixed latency units which are: ALU, Branch unit, CSR buffer, Multiplier and Division. The division is the only one that's not fixed. The multiplier lasts 2 cycles and the CSR buffer blocks the write back until this instruction has been committed. As for the other FUs, they have no particularity and last one cycle. The next two write back ports are for the load and store units, coming from the load store unit (LSU). The fourth port is only synthesized if the floating point operations are enabled or if the superscalar configuration is enabled. In one case it writes back the floating point unit results and in the other the second ALU results. It is worth noting that the FPU is not officially supported in superscalar mode (issue_read_operands.sv line 875). Finally the last write back port is only synthesized if a coprocessor is connected to either the accelerator or the CV-X-IF channel. Both cannot be instantiated at the same time since they share the same write back port. Moreover the accelerator is not officially supported in superscalar mode as stated in the original codebase (cva6.sv line 1850).

### C. Load Store Unit (LSU)

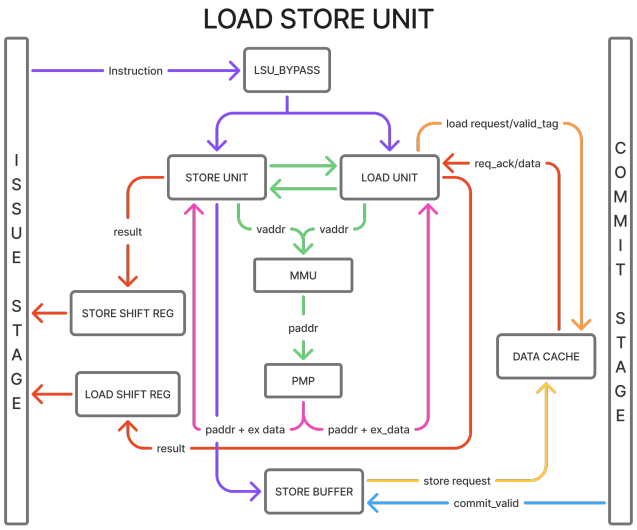



Fig. 1. LSU instruction flow

This is the final module of the original design that will be detailed. It handles all the memory operations. It is composed of multiple submodules: **Memory Management Unit** (MMU), **Physical Memory Protection** (PMP), **lsu_bypass**, load unit, store unit and a store buffer. In order to understand what really changed from one design to another we will need to detail the instruction flow in the LSU. First, instructions are added to the **lsu_bypass** which is a simple First In First Out buffer. One instruction is dispatched per cycle with fall-through enabled. When an instruction is dispatched, the corresponding unit starts the translation by sending the virtual address to the MMU. From that point the behavior differs between the load and store unit. First we will take a look at the store unit. It simply waits for the response of the PMP and writes back immediately upon reception. If an exception has occurred the instruction is not added to the store buffer. Otherwise it is added to the speculative queue of the store buffer. When a commit signal is received, the instruction is transferred from the speculative queue to the commit queue and the request is sent to the cache in order. As for the load unit it is a bit more complicated. The data cache is virtually indexed, physically tagged (VIPT): the index used to select the cache set only uses bits that are inside the page offset, and these bits are the same in the virtual and in the physical address. So the load unit sends the index to the cache at the same time as the start of the translation, and the cache can start reading the set. It is an optimization to gain 1 cycle. When the physical address is received, if an exception has occurred we write back the exception data and cancel the request. Otherwise the tag of the physical address is sent to the cache which completes the access. If the translation is not ready (TLB miss) the request is also cancelled and replayed later. When the data is received we write it back the same cycle. For both units the write back goes through a shift register. It helps with lengthy data paths. The store shift register is usually disabled because the timing does not require it. However the load shift register is necessary because traversing the cache and the execute stage all the way to the issue stage and scoreboard, to then perform forwarding, is not possible due to timing constraints. It is worth noting that even on bare metal, when no MMU is synthesized, to keep compatibility between different configurations, instructions have to spend one cycle in the MMU. Finally the load and store units also communicate in order not to send a load to the cache if a store needs to write to that same address before. It only blocks the load and does not perform any sort of store-to-load forwarding.

## III. Out of Order Design

In this section we will go over the implementation of the Out of Order execution. First we will explain how the different modules developed work and then how they are linked by looking at the global design. There are 3 new modules: a **register allocation table** (RAT), **reservation stations** (RS) and a **load store queue** (LSQ). There is also the **Reorder Buffer** (ROB) which reuses a lot of the old scoreboard code.

### A. Register Allocation Table

The register allocation table renames instructions sent from the decode stage. It keeps a table with the correspondence between architectural registers and physical registers as well as a list of available registers. Each instruction is given a new destination register picked among the available registers and all the operands are renamed by the physical register holding the real value of the architectural one. There are 2 types of RAT, the commit RAT and the speculative RAT. The speculative one is updated speculatively every time an instruction is renamed. The commit RAT is updated only when an instruction is committed. It allows us to restore the RAT state in one cycle after an exception by simply replacing it by the commit RAT data. It is a very simple module. However there is one important optimization used to reduce the critical path. In superscalar mode, the RAT uses 2 priority encoders in cascade to elect 2 free registers that will be used as a destination for the renamed instructions. This step is registered. The two registers are elected using the registered correspondence table minus the registers allocated in the current cycle. Then, the next cycle, the elected registers are either the currently selected ones from the registered correspondence table or the previously selected ones if they were not used for some reason (for example the ROB is full). This optimization evicts the priority encoder cascade from the issue stage critical path. The only downside is that a freed register becomes available 1 cycle later.

### B. Reservation Station

The reservation station (RS) tracks all the dependencies for each instruction inside it. Each operand of each instruction

has a bit attached saying if it is available or not. Validity is updated in various ways. The first one is using a table called **is_result_available**, updated each cycle within the issue stage and sent to all the units in need of it. This structure keeps track of which physical register has a valid value. However it is registered so it takes one cycle to update and then one cycle for the RS to update its dependencies. This is why we also update the dependencies based on write backs received this cycle by snooping the **Common Data Bus** (CDB). We also check RAW dependencies between the 2 newly renamed instructions in the superscalar configuration. And finally, we implemented an early wakeup only for fixed latency single cycle instructions. This means we update dependencies based on currently dispatched instructions. For example, if an add is issued, we know its result will be ready next cycle so we already mark it as such. It is very useful: since the dependency state of each operand is registered, we gain one cycle for each fixed single cycle latency operation. All these updates are computed in parallel and in the end a final mux selects the right value for each operand and each entry in the reservation station with a priority: RAW updates > early wakeup > write back > is_result_available. In order to add an instruction to the RS we use a cascade of two priority encoders like in the RAT. And to choose which one to issue we use one tournament tree to compare the id of each instruction and elect the oldest ready one. This module also has a **fall-through** mode in which an instruction renamed this cycle can be dispatched the same cycle only if is_result_available already marks all its operands as ready. It lengthens the critical path by a small margin but gains performance.

### C. Reorder Buffer

The ReOrder Buffer (ROB) is a central data structure which is similar to a scoreboard but with more features. Like the scoreboard it holds status information for every instruction and commits them in order. But in Out of Order we need to store more information and also have access to more information. For example, in our design we have a number of physical registers greater than the number of architectural registers. And thanks to the renaming we can write data into the registers as soon as the data is produced whereas in the original design results are written during the commit stage. This change means we need new logic in the ROB to extract the destination register for each write back port. But the real new job of the ROB is to control the **rollback**. In an Out of Order pipeline, if a branch misprediction occurs we have to roll back to a state before the branch and undo the changes made by the speculative instructions as well as squash the speculative instructions still executing. The ROB is the module that pilots this signal. It sends a signal to the controller which stalls the front-end (PC, Fetch, Decode) and then sends the rollback information to all the modules in need of it. This includes the RS, LSQ_bypass, RAT, LSQ as well as all functional units. This also means the ROB needs to store information such as the architectural destination register of each instruction or the previous physical register holding the real value of the architectural register.

### D. Load Store Queue (LSQ)

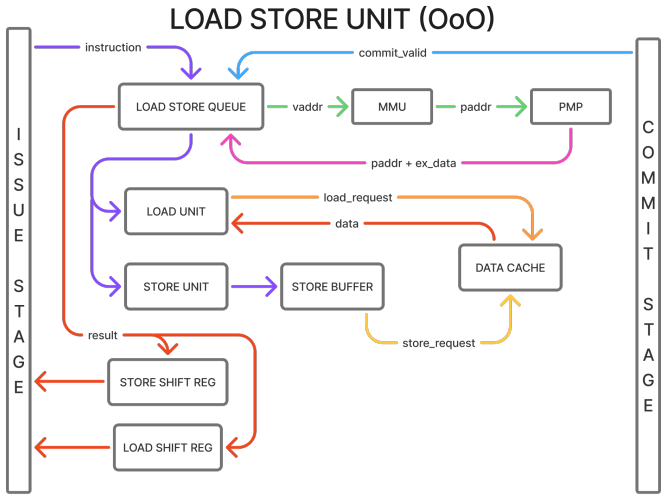



Fig. 2. OoO LSU instruction flow

This is the biggest module implemented for this design so we will spend more time explaining the design choices. It is instantiated inside the LSU and replaces the lsu_bypass as well as some functionality previously handled by the load/store units. Let's break down the new instruction flow inside the LSQ and LSU. First, the LSQ can receive up to two instructions per cycle in superscalar configuration. When instructions are received, they are added to either the **load queue** or the **store queue** which are 2 distinct data structures of equal size. The store queue is a circular buffer whose pointers rely on natural binary wrap-around, which restricts its size to powers of two. Store commit being in order allows us to use such a data structure. As for the load queue, since loads can be retired from the queue out of order, we cannot do the same. It is managed using a free list and a cascade of two priority encoders to elect a free entry. Thus it is not required to be a power of two. But the current design has only one parameter shared between the 2 queues. Instructions dispatched to the LSQ can either have their data valid (virtual address for loads and stores and data to store only for stores) or not. If they do not have all their data valid then the LSQ snoops the CDB for them starting the same cycle that the instructions are being dispatched. Then, during the same cycle, we elect the instruction to be sent to the MMU. This means that for each instruction (including the ones dispatched this cycle) we check the virtual address validity and, using an **age matrix**, we compare the age of the instructions and elect the oldest one. Then, when the virtual address is translated, it goes through the PMP which is only combinational and verifies potentially unauthorized accesses. During this cycle, we verify address dependencies between stores and loads, and from now on the behavior varies between store operations

and load operations. Store operations always write back at this moment as in the original design. If an exception has occurred the instruction is flagged as invalid and the store queue entry is freed. If no exception occurs it is flagged as valid and will be sent to the store unit in order when a store commit is received. The speculative queue in the store buffer has been removed so it is immediately inserted in the commit queue of the store buffer. This also means that the store unit is basically useless as it does nothing meaningful. But it was kept for simplicity. Stores are freed from the LSQ only when they have been successfully sent to the cache. We cannot release them before without missing potential address dependencies between loads and stores. Now let's take a look at the load instruction flow. If an exception occurs they will be flagged as result ready and the result will be written back next cycle (here the result is the exception information). If no exception occurred, we verify address dependencies with all older stores, perform store-to-load forwarding if necessary (only with same width operations), block the instruction in case of partial address match and, finally, if no match has been found or not all previous stores have their address translated we flag this load as valid. We then send the oldest load to the load unit. This new flow implies that the old optimization where the cache index is sent before the translation is no longer used. We even suppressed the handshake with the HPD cache to gain one cycle since we don't need the handshake anymore. When the result is received we store it in the LSQ and send the result the next cycle. This mechanism replaces the load shift register which is now disabled in OoO. If a load has been sent to the load unit before all previous stores had a valid physical address, we cancel it if a dependency is found later. This also means we do not write back until all dependencies have been resolved. But this happens extremely rarely since address translation is done by starting with the oldest instruction.

## IV. Global overview

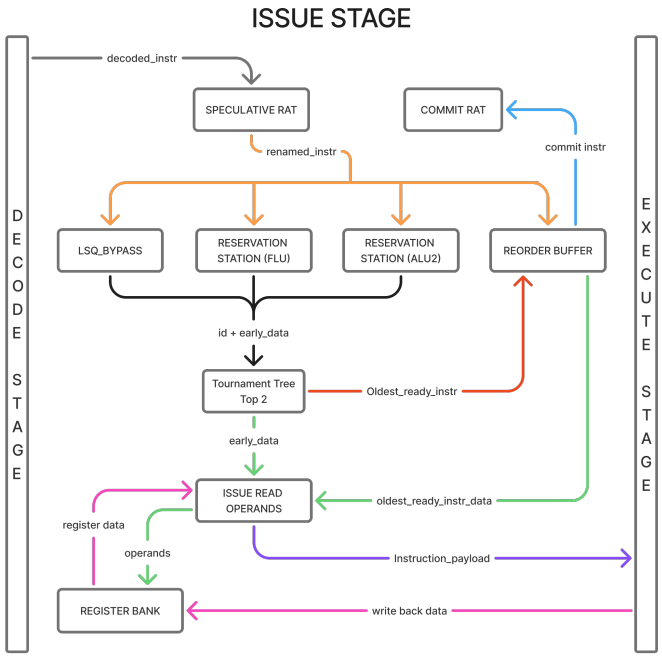



Fig. 3. Issue stage instruction flow

Now that we have seen in detail all the OoO modules, we will discuss the global design. You can see above a simplified version of the issue stage. It lacks some small details but retains the important flow and stays simple. First, instructions are issued from the decode stage. They are immediately renamed by the speculative RAT. Renamed instructions are then sent to the RS, ROB and LSQ_BYPASS where they are added to the ROB if it is not full and to the corresponding RS or to LSQ_BYPASS in case of memory operations. There is one RS (RS0) for the fixed latency units write back port and one for the ALU2 (RS1) in superscalar configuration. It was done so each reservation station can elect one instruction for its write back port and maximize the usage in dual issue while maintaining a healthy timing by not having one big RS. But this means we have to ensure a relatively even dispatch of instructions between the two RS, which both can store ALU operations. We do this by always adding ALU operations to the RS1 first. Then if there is another ALU instruction the same cycle we store it in the RS0. This simple method proved itself very effective, even more than a version optimized for an even distribution between the RS. As for the rest of the write back ports, the LSQ_BYPASS plays the role of a stripped down reservation station. It is a simple module that stores memory operations in order and shows them as candidates to the top-2 tournament tree, also in order. It possesses a fall-through mode just as the RS but unlike them it can present up to two instructions per cycle to the tournament. The protocol we chose to issue instructions to the load store queue is **token based** and works differently for stores and loads. For loads the number of tokens is updated based on write backs and rollbacks. Meaning the token counter is updated combinatorially in the same cycle

TABLE I

PARAMETERS, AREA (CORE LUTS AFTER OUT-OF-CONTEXT PLACE AND ROUTE) AND MAXIMUM FREQUENCY ($T_{min}$ = CONSTRAINED PERIOD − WNS) OF EACH VERSION. SB: SCOREBOARD ENTRIES (IN-ORDER); ROB: REORDER BUFFER ENTRIES (OOO). RS, LSQ AND BYPASS GIVE THE NUMBER OF ENTRIES OF EACH STRUCTURE.

| Version | Phys. reg. | RS | LSQ | Bypass | Core LUTs | vs. ref. | Constraint (ns) | $T_{min}$ (ns) | $F_{max}$ (MHz) |
|---|---|---|---|---|---|---|---|---|---|
| In-order 1-way SB16 | — | — | — | — | 14 381 | ref. | 20 | 20.11 | 49.7 |
| In-order 1-way SB8 | — | — | — | — | 10 933 | –24.0% | 15 | 19.63 | 51.0 |
| In-order 2-way SB16 | — | — | — | — | 23 269 | +61.8% | 22 | 23.53 | 42.5 |
| In-order 2-way SB8 | — | — | — | — | 18 410 | +28.0% | 18 | 18.81 | 53.2 |
| OoO 1-way ROB16 | 48 | 8 | 4 | 4 | 21 838 | +51.9% | 21 | 22.40 | 44.6 |
| OoO 1-way ROB8 | 40 | 4 | 2 | 2 | 16 344 | +13.6% | 18 | 19.60 | 51.0 |
| OoO 2-way ROB16 | 48 | 4 | 4 | 4 | 33 794 | +135.0% | 22 | 23.71 | 42.2 |
| OoO 2-way ROB8 | 40 | 3 | 2 | 2 | 25 261 | +75.7% | 20 | 23.88 | 41.9 |

and then also used the same cycle. Once the token count is updated, we only add a load to LSQ_BYPASS if a token is available. The potential issue is that a load would block the front-end because of a lack of tokens while there is still space in LSQ_bypass. But our measurements showed this particular scenario frequency only ranged from 0% to 4.4% of the time depending on the benchmark across all configurations. Plus this choice was made to allow the use of a combinatorially updated token counter. Otherwise it would not fit in the tight timing constraint of the instruction selection path. As for store operations, the token update is done upon reception of a signal from the LSQ because only this module knows when a store has successfully been sent to the cache. The update is registered in the LSQ as well as in the issue stage to cope with the lengthy critical path. And unlike the load insertion, the token number is decreased only when a store has been elected to be dispatched. Thus, we do not have the same issue as with the load tokens. In addition to these data structures, as mentioned previously, whenever an instruction is renamed and successfully added to the corresponding units, a small buffer named **is_result_available** updates which physical register has a valid value. It is crucial to verify dependencies in the reservation stations and to determine if an instruction sent to the LSQ already has valid data or a valid address, or if it needs to be snooped by this module. After all modules have sent their candidates to the tournament, 2 instructions are elected and sent to IRO. A part of the data required to form the data payload is sent from LSQ_BYPASS or the RS and the rest is fetched from the ROB. This separation helps reduce the critical path by not waiting for the ROB data to compute all the early information in IRO. Finally another advantage of this architecture is that the renaming ensures that the destination register of an instruction is no longer needed. This means we can write into the register bank as soon as the result is ready and remove the old forwarding system. We replace it by a very simple comparison between the registers to write this cycle and the registers to read the same cycle.

## V. RESULTS AND ANALYSIS

We will now present the results obtained with this design through various configurations. We will discuss IPC gains, LUT consumption, and timing. The design has been synthesized on a Digilent Zybo Z7-20 (Xilinx Zynq-7020) using the tools provided in the Thales CVA6 softcore contest GitHub repository [2]. Except for the OoO parameters, the rest of the CPU configuration is identical across all versions. It is in 32-bit mode with no FPU and no MMU but with non-idempotent memory enabled, and the HPD cache is used. As for the OoO parameters, here is the list:

- Number of physical registers
- Reservation stations size
- Number of rolled back instructions per cycle
- LSQ_BYPASS size
- LSQ size (same for the load queue and store queue, and must be a power of 2)
- Fall-through for RS and LSQ_BYPASS

As for the parameters used for the configurations tested, it was decided mainly by the synthesis results and the usefulness. But we did have some guidelines that helped pick the size of each data structure. For the number of physical registers we followed the rule: number of architectural registers + size of the ROB (the number of architectural registers being 32 on RISC-V). For the RS we chose half of the size of the ROB. So for example with a ROB of 16 entries we will have 2 RS of size 4. The size of the rollback is arbitrarily set to 2. The LSQ_BYPASS and LSQ are set to the same size as the RS. And the fall-throughs are all enabled. Finally the measurements are done using a ROB size of 8 and 16 to see how the increased context can help the out of order execution. A recap of the configurations is shown here (Table I) along with some design information.

All the configurations are tested on 3 benchmarks that show different types of workload. The fast Fourier transform (FFT) which is mainly computations with very few big chains of dependencies. A small convolutional neural network (MNIST) which relies heavily on memory operations. And finally CoreMark which contains a lot of branches. The performance data are measured on the FPGA after flashing the design and all benchmarks have been compiled with GCC 15.2.0. The timing data are measured with Vivado reports in out of context mode with the same parameters, and LUT consumption is measured with the post-implementation report from Vivado for the whole CPU numbers (Table VI) but in out of context mode for the core numbers (all the other figures). Finally the performance counters are measured in simulation using Questa. Throughout this analysis all configurations will be mentioned but we will mainly focus on the superscalar OoO configuration with a ROB16 which is the subject of this

TABLE II

SPEEDUP OVER THE REFERENCE (IN-ORDER, SINGLE ISSUE, SB16). "EACH AT ITS $F_{\max}$" MULTIPLIES THE CYCLE SPEEDUP BY THE RATIO OF MAXIMUM FREQUENCIES; "PERF. / LUT" DIVIDES THE GEOMETRIC MEAN BY THE AREA RATIO.

| Version | At equal frequency | | | | Each at its $F_{\max}$ | Perf. / LUT |
|---|---|---|---|---|---|---|
| | FFT | MNIST | CoreMark | Geomean | | |
| In-order 1-way SB16 | 1.00× | 1.00× | 1.00× | **1.00×** | 1.00× | 1.00 |
| In-order 1-way SB8 | 1.00× | 1.00× | 1.00× | **1.00×** | 1.02× | 1.32 |
| In-order 2-way SB16 | 1.23× | 1.34× | 1.24× | **1.27×** | 1.08× | 0.78 |
| In-order 2-way SB8 | 1.23× | 1.33× | 1.24× | **1.27×** | 1.36× | 0.99 |
| OoO 1-way ROB16 | 1.09× | 1.36× | 1.11× | **1.18×** | 1.06× | 0.78 |
| OoO 1-way ROB8 | 0.95× | 1.12× | 1.05× | **1.04×** | 1.06× | 0.91 |
| OoO 2-way ROB16 | 1.43× | 2.16× | 1.52× | **1.68×** | 1.42× | 0.71 |
| OoO 2-way ROB8 | 1.10× | 1.31× | 1.28× | **1.23×** | 1.04× | 0.70 |

article, the most effective configuration and the focus of the development.

### A. Instructions per cycle (IPC)

First let's take a look at the results we obtained from an IPC gain standpoint. You can see the raw IPC of each benchmark for each configuration as well as their speedup compared to the base configuration here (Table VII and Table II). All the OoO versions perform better than their in-order counterpart on all benchmarks except for the superscalar version with a ROB size of 8 (Table VIII). This version is heavily bottlenecked by the number of physical registers (between 3% and 20% of the time as shown here Table IV). This is because the formula we used to compute the number of physical registers needed does not take into account the pre-allocation in the RAT as well as the latency of liberation. For the registers to never be an issue we would need to have 32 + ROB + 4 physical registers. We measured the impact of increasing the number by 4 (Table IX). The results are modest because the bottleneck has now shifted to other structures, mainly the LSQ. This problem is shared between the two ROB 8 configurations. The small window on all data structures does not allow extracting the parallelism of the running program. As for the single issue configuration with a ROB of size 16 shows good gains for all benchmarks. FFT reaches 0.837, CoreMark 0.779 and MNIST even reaches 0.891 IPC. For a single issue configuration with a maximum theoretical IPC of 1 it is a good result. We are mostly bottlenecked by the single issue.

However, in some cases, the OoO configuration can even perform worse than the in-order version. When the data structure sizes do not allow for parallelism, not only do we see no gain but the downsides of the OoO architecture, such as rollback or the latency of dependency updates (load-to-use latency for example), can add up and ruin the potential gains. But there is one configuration performing best by far: the dual issue ROB 16 superscalar, by a factor of 1.32 when compared to its in-order counterpart (Table VIII). We will now take a closer look at this configuration and try to explain the results.

The OoO gain is set by how badly the in-order core handles their latencies and by which machine resource they hit first. So let's analyze the benchmarks (Table III). FFT is already performing very well in the in-order single issue configuration with an IPC of 0.77. This is because 40% of the operations had their sources computed more than 16 instructions earlier.

TABLE III

CHARACTERISATION OF THE BENCHMARKS FROM THE COMMITTED INSTRUCTION STREAM (INDEPENDENT OF THE CORE). IDEAL ILP ASSUMES UNLIMITED ISSUE WIDTH, PERFECT BRANCH PREDICTION, FIXED LATENCIES (ALU 1, LOAD 2, MUL 2 CYCLES) AND IGNORES DEPENDENCES THROUGH MEMORY; IT IS AN UPPER BOUND.

| | FFT | MNIST | CoreMark |
|---|---|---|---|
| *Instruction mix (% of instructions)* | | | |
| Conditional branches | 3.0% | 3.4% | 17.9% |
| Jumps (jal / jalr) | 1.0% | 0.1% | 2.2% |
| Loads | 17.4% | 45.5% | 20.9% |
| Stores | 13.7% | 0.4% | 5.2% |
| MUL | 6.4% | 21.8% | 3.4% |
| ALU | 58.5% | 28.9% | 50.4% |
| *Branch prediction* | | | |
| Mispredictions / 1000 instr. | 2.5 | 3.8 | 20.6 |
| Mispredicted branches | 8.2% | 11.3% | 11.6% |
| Instr. between mispredictions | 403 | 261 | 48 |
| *Producer → consumer distance (% of sources)* | | | |
| 1 instruction | 15.1% | 5.6% | 16.2% |
| 2 | 15.6% | 14.5% | 19.1% |
| 3–4 | 7.2% | 23.3% | 16.7% |
| 5–8 | 10.0% | 11.5% | 18.4% |
| 9–16 | 11.9% | 16.9% | 12.1% |
| > 16 | 40.2% | 28.2% | 17.5% |
| *Ideal dataflow ILP (window of $W$ instructions)* | | | |
| $W = 8$ | 2.98 | 3.28 | 2.85 |
| $W = 16$ | 3.98 | 3.97 | 3.60 |
| $W = 32$ | 5.76 | 4.60 | 4.25 |
| $W = \infty$ | 149 | 2 066 | 109 |

Thus, the in-order core rarely stumbles upon short chains of dependencies. The dual issue mode and the OoO design extract a little more IPC. But the OoO version suffers from a big store bottleneck. This benchmark has 13.7% of stores and stores wait for a token 40% of the time. Doubling only the LSQ raises FFT IPC by 4.1% and store credit stalls fall from 40% to 10% of cycles, but the bottleneck moves to the ROB, blocked by stores waiting for the single-store-per-cycle drain. This means that the design is probably restricted by the single issue cache. A dual issue cache could probably significantly improve the performance.

The next benchmark is MNIST. This is by far the best performing, mainly because it is memory intensive (45.5% of loads). This is both the worst in-order scenario and the best Out of Order scenario. In OoO we can exploit the load latency

TABLE IV

CYCLE BREAKDOWN IN SIMULATION (% OF MEASURED CYCLES; COMMIT IPC). DECODE STALL CAUSES ARE EXCLUSIVE AND CHECKED IN PRIORITY ORDER ON PORT 0. [a] LSQ_BYPASS FULL OR NO LOAD CREDIT AVAILABLE. THE 2-WAY ROB32 VERSION IS THE SCALING STUDY OF TABLE XIII (EVERY STRUCTURE DOUBLED; SIMULATION ONLY). DECODE-SIDE COLUMNS (FRONT-END EMPTY AND DECODE STALL CAUSES) ARE MUTUALLY EXCLUSIVE. NOTHING READY IS MEASURED AT ISSUE AND MAY OVERLAP WITH THEM, TYPICALLY WITH FRONT-END EMPTY AFTER A MISPREDICTION.

| OoO | Bench. | IPC | Dual issue | Nothing ready | Front-end empty | Decode stalled by | | | | LQ full | SQ full |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | Phys. reg. | ROB full | RS full | LSQ[a] | | |
| 1-way ROB16 | FFT | 0.883 | 0.0 | 10.9 | 3.7 | 0.1 | 0.8 | 0.6 | 5.9 | 1.1 | 6.8 |
| | MNIST | 0.907 | 0.0 | 8.4 | 4.0 | 0.0 | 0.0 | 0.0 | 4.2 | 2.9 | 0.0 |
| | CoreMark | 0.815 | 0.0 | 15.3 | 13.7 | 0.0 | 0.0 | 0.2 | 0.9 | 0.4 | 0.2 |
| 1-way ROB8 | FFT | 0.771 | 0.0 | 22.7 | 1.7 | 1.9 | 1.0 | 0.5 | 17.6 | 13.2 | 24.3 |
| | MNIST | 0.745 | 0.0 | 25.0 | 1.6 | 1.4 | 0.0 | 0.0 | 22.0 | 25.9 | 0.0 |
| | CoreMark | 0.768 | 0.0 | 20.0 | 10.1 | 0.4 | 0.7 | 1.4 | 7.0 | 9.8 | 2.9 |
| 2-way ROB16 | FFT | 1.278 | 50.2 | 16.6 | 2.6 | 0.4 | 8.2 | 7.0 | 9.2 | 3.2 | 16.7 |
| | MNIST | 1.453 | 57.1 | 8.6 | 3.5 | 2.4 | 0.0 | 0.0 | 8.9 | 10.4 | 0.0 |
| | CoreMark | 1.162 | 45.8 | 17.7 | 17.2 | 0.1 | 1.3 | 3.3 | 3.5 | 3.8 | 3.2 |
| 2-way ROB8 | FFT | 0.930 | 28.2 | 31.1 | 1.7 | 15.1 | 3.4 | 0.0 | 19.2 | 22.6 | 31.2 |
| | MNIST | 0.873 | 20.0 | 31.4 | 2.0 | 19.4 | 0.0 | 0.0 | 20.6 | 34.9 | 0.1 |
| | CoreMark | 0.965 | 31.4 | 27.1 | 10.8 | 3.2 | 6.2 | 1.8 | 12.6 | 17.3 | 5.3 |
| 2-way ROB32 | FFT | 1.460 | 64.8 | 10.8 | 5.6 | 0.0 | 7.9 | 1.4 | 5.8 | 2.7 | 11.4 |
| | MNIST | 1.595 | 69.5 | 6.3 | 6.4 | 0.0 | 0.0 | 0.0 | 5.0 | 4.4 | 0.0 |
| | CoreMark | 1.175 | 48.6 | 20.1 | 22.1 | 0.0 | 1.0 | 1.4 | 0.2 | 0.4 | 2.7 |

TABLE V

OOO DUAL ISSUE (ROB16) AGAINST THE CHEAPEST IN-ORDER DUAL-ISSUE VERSION (SB8, SAME PERFORMANCE AS SB16 WITHIN 0.2%). A POSITIVE DIFFERENCE FAVOURS THE OOO VERSION, EXCEPT ON COST ROWS (LUTS, $T_{\min}$) WHERE IT IS THE INCREASE. [a] EARLY LOAD WAKEUP (TABLE XV): NOT RUN ON THE BOARD, ITS CYCLES ARE ESTIMATED FROM THE MEASURED ROB16 CYCLES SCALED BY THE IPC RATIO OBSERVED IN SIMULATION.

| | In-order 2-way SB8 | OoO 2-way ROB16 | Diff. | OoO ROB16 + EW[a] | Diff. |
|---|---|---|---|---|---|
| *Hardware cost* | | | | | |
| Core LUTs | 18 410 | 33 794 | +83.6% | 33 573 | +82.4% |
| $T_{\min}$ (ns) | 18.81 | 23.71 | +26.0% | 23.97 | +27.4% |
| $F_{\max}$ (MHz) | 53.2 | 42.2 | −20.7% | 41.7 | −21.5% |
| *Performance at equal frequency* | | | | | |
| FFT IPC | 0.949 | 1.101 | +16.0% | 1.108 | +16.7% |
| MNIST IPC | 0.873 | 1.415 | +62.1% | 1.428 | +63.6% |
| CoreMark IPC | 0.865 | 1.064 | +23.0% | 1.083 | +25.1% |
| Geomean | — | — | **+32.3%** | — | **+33.7%** |
| *Execution time, each at its $F_{\max}$* | | | | | |
| FFT (µs) | 2 193 | 2 382 | −8.0% | 2 393 | −8.4% |
| MNIST (µs) | 24 133 | 18 760 | +28.6% | 18 796 | +28.4% |
| CoreMark (µs) | 17 917 | 18 359 | −2.4% | 18 244 | −1.8% |
| Geomean | — | — | **+4.9%** | — | **+4.9%** |
| Perf. at $F_{\max}$ / LUT | — | — | −42.8% | — | −42.5% |

and make use of the short chains of instruction dependencies (about 38% between 2 and 4 instructions of length) where the in-order core struggles a lot. This benchmark ends up as the one with the best IPC by a fair amount with 1.4 and few visible bottlenecks due to the data structure size. The only visible one is the load queue being full about 10% of the time. CoreMark is the last benchmark and shows a different picture. It is heavily branched (17.9% of branches) and with an 11.5% misprediction rate we have one misprediction every 49 instructions. This penalizes our CPU not only because of the multicycle rollback (about 2 cycles of rollback on average) but we also have to refill the ROB and all the data structures again to make use of the OoO design. So the front-end and the branch prediction end up being the biggest bottleneck of this benchmark.

Now let's compare this configuration to the in-order superscalar ones. The first thing we notice is that the base CVA6 has the same performance regardless of the scoreboard size so we will ignore the configuration with a scoreboard of size 16. In Table V we can clearly see the performances compared at the same clock frequency and then at their own maximum clock frequency. The results are rather mixed. When compared at the same clock frequency the OoO clearly outperforms the in-order core by an average of **+32%**. But when compared each at their own clock frequency, the average drops down to only **+5%**. The MNIST benchmark still shows great gains with +28.6% but both FFT and CoreMark are at a loss.

However, these benchmarks do not show the biggest strength of the OoO, the ability to hide the cache miss latency. They all fit entirely in the data cache so this is never observed. We made a short program to test this with 3 functions iterating

TABLE VI

LUTs per pipeline stage of the dual-issue cores (hierarchical report of the full SoC implementation); the change is computed against the in-order version with the same queue size (SB16). Indented lines detail the stage above. Vivado optimises across module boundaries, so logic can be counted in a neighbouring block (in the SB8 run, the decoder is absorbed by the issue stage); the back-end subtotal is the most reliable comparison. Blocks add up to slightly more than the core total because LUTs shared by two instances are counted in both. [a] Including their tournament trees. ≫: block almost empty in the reference.

| Block | In-order SB8 | | In-order SB16 | | OoO ROB16 | | OoO vs. In-order SB16 | | |
|---|---|---|---|---|---|---|---|---|---|
| | LUTs | % core | LUTs | % core | LUTs | % core | Ratio | LUTs | % of increase |
| **Front-end** | 3 427 | 18.8% | 3 345 | 14.9% | 3 245 | 10.1% | 0.97× | −100 | −1.0% |
| **Decode (`id_stage`)** | 8 | 0.0% | 1 212 | 5.4% | 1 274 | 4.0% | 1.05× | +62 | 0.6% |
| **Issue stage** | 9 011 | 49.4% | 9 668 | 43.1% | 15 048 | 47.0% | 1.56× | +5 380 | 55.9% |
| Scoreboard / ROB | 2 671 | 14.6% | 4 885 | 21.8% | 6 967 | 21.7% | 1.43× | +2 082 | 21.6% |
| RAT (speculative + commit) | — | — | — | — | 1 657 | 5.2% | new | +1 657 | 17.2% |
| Reservation stations[a] | — | — | — | — | 1 329 | 4.1% | new | +1 329 | 13.8% |
| `lsq_bypass` | — | — | — | — | 534 | 1.7% | new | +534 | 5.6% |
| Read operands + regfile | 6 341 | 34.7% | 4 783 | 21.3% | 4 568 | 14.3% | 0.96× | −215 | −2.2% |
| **Execute stage** | 1 446 | 7.9% | 3 769 | 16.8% | 8 763 | 27.4% | 2.33× | +4 994 | 51.9% |
| LSQ / `lsu_bypass` | 212 | 1.2% | 297 | 1.3% | 7 330 | 22.9% | 24.68× | +7 033 | 73.1% |
| Load, store units, PMP | 554 | 3.0% | 2 618 | 11.7% | 702 | 2.2% | 0.27× | −1 916 | −19.9% |
| MUL / DIV | 281 | 1.5% | 456 | 2.0% | 376 | 1.2% | 0.82× | −80 | −0.8% |
| CSR buffer | 401 | 2.2% | 398 | 1.8% | 394 | 1.2% | 0.99× | −4 | 0.0% |
| **CSR register file** | 1 094 | 6.0% | 1 140 | 5.1% | 873 | 2.7% | 0.77× | −267 | −2.8% |
| **Caches (I$ + D$)** | 3 505 | 19.2% | 3 522 | 15.7% | 3 129 | 9.8% | 0.89× | −393 | −4.1% |
| *Back-end (decode + issue + execute)* | 10 465 | 57.3% | 14 649 | 65.3% | 25 085 | 78.3% | 1.71× | +10 436 | 108.5% |
| **Core total** | **18 252** | 100% | **22 420** | 100% | **32 039** | 100% | **1.43×** | **+9 619** | 100% |
| Flip-flops | 8 192 | — | 9 479 | — | 9 905 | — | 1.04× | +426 | — |

over an array bigger than the data cache. The first function iterates over the array by steps of 64 bytes and thus reads one element per cache line (stream). This causes regular independent cache misses. The second function does a sum but using precomputed addresses chosen seemingly at random (gather). This causes irregular independent cache misses. The last one always uses the data of the previous load (pointer chasing). The OoO core ends up **1.62×** faster in cycles (**1.28×** at Fmax) when cache misses are independent (stream, gather), because it overlaps several misses. When each address depends on the previous load (pointer chasing), it is 6% slower, paying a longer load-to-use latency. To sum it up, the OoO core clearly outperforms the in-order core in benchmarks where data latency and cache misses play a big role, which is exactly the main purpose of this architecture.

### *B. LUT consumption*

After analyzing the performance we also have to take into account the design area. The OoO core uses significantly more LUTs in every configuration. To identify where the main source of LUT consumption is, we will take a closer look at the changes between In order and Out of Order shown here (Table VI). The LSQ alone represents about 52% of the global LUT increase compared to the In Order SB8. It is no surprise, it is full of associative structures that map poorly to LUTs. The Scoreboard/ROB also sees a big increase. In the original design, few accesses to its data were necessary. The accesses were needed to perform write back, forwarding or issue/dispatch/commit instructions. But now the rollback needs to iterate over the entire structure and the write back also needs to access the destination registers to write into the right register. This added logic has a cost. It is probably the part that could benefit the most from a refactor. Some data are never accessed multiple times. So a portion of the ROB could be turned into LUTRAM to save some space. Finally the reservation stations, LSQ_bypass and register allocation tables are pretty cheap.

Overall, in the superscalar ROB16 configuration, the design can present small congestion spots, especially around the LSU. The design when fully implemented uses 42,099 LUTs out of the 53,200 LUTs available on the Zybo Z7-20. And the OoO data structures now represent 55.6% of the core (Table X).

It is clear that the LUT consumption is a lot bigger, especially when comparing the ROB 16 OoO superscalar with the in-order SB 8 superscalar (Table V). It is a fair comparison since the in-order version has the same performance with a scoreboard equal to 8 or 16. If we account for Pollack's rule, which states that the performance increase from micro-architectural improvements is roughly proportional to the square root of the increase in complexity (processor logic area), the result is to be expected. The OoO superscalar ROB16 configuration consumes 1.84 times more LUTs than the in-order superscalar configuration. The square root of 1.84 is approximately 1.36 which is almost the same as the geometric mean we obtained (1.323). Plus we have to bear in mind that the processor was synthesized on an FPGA. Data structures such as CAM memories, which are heavily used in the LSQ, and associative structures in general that are used throughout the design are way more optimized on ASIC than on FPGA and thus consume less logic area.

### *C. Timing analysis*

We already established that the design has good performance, with a reasonable area increase, but is held back by its

clock frequency. So let's see why the timing is so tight. First, let's take a look at the critical path of the original design. The single issue SB8 configuration starts in the scoreboard, commits an instruction then ends in it. As for the other configurations, they always start from the PMP and can end, for example, in lsu_bypass to validate the dispatch to the load unit in the case of single issue mode. Or, for the superscalar version, it can end in the scoreboard where it validates or not the addition of a new instruction as well as compares the operands of the newly added instruction. This is because the PMP response triggers a write back the same cycle for stores. Note that this path is actually not necessary. The write back is necessary to validate a commit and send exception data to the scoreboard but we do not need to compare operands since a store does not return any value. But this is not an issue for us since we will be comparing the OoO version to the in-order version with a scoreboard size equal to 8. And the critical path previously mentioned only appears because of the scoreboard size and the increased number of comparators due to the dual issue mode. Now let's take a look at the OoO version critical path. The single issue ROB8 configuration critical path is similar to the original design, it starts from the PMP and ends in the cache. It can be explained because it only needs one small RS and a small LSQ_bypass as well as fewer comparators due to the single issue mode. And for the single issue ROB16 configuration the critical path becomes the bypass translation -> same cycle issue in the LSQ and ends in the cache. For all superscalar versions, the critical path is the selection of the next instruction to dispatch in the issue stage and varies very little in length with RS size changes or ROB size changes. As shown by the superscalar version with ROB8 which has the same timing as the one with ROB16 but with every data structure length divided by two (the slight increase in data path is very likely due to Vivado run-to-run variance and the fact that the design is very congested). It is to be expected from our design: the selection of an instruction becomes an essential part of the pipeline, especially with fall-through enabled. The instruction has to be renamed, go through the dependency check, the tournament tree and read the registers as well as perform forwarding, all in one cycle. Now let's take a closer look at the superscalar version with a ROB size of 16 which is the subject of this article. We can try to shorten this path by reducing the size of the RS or LSQ_bypass. But it would only hurt performance, at least for the reduction of the RS size. As shown here (Table IV) the RS can end up full for up to 7% of the time. Moreover the gain in clock time would be either non-existent, as shown by the ROB8 configuration critical path, or negligible because of the second critical path of the design. When the fall-through is completely disabled, the critical path becomes the translation bypass and issue in the same cycle to the load unit (Table XI).

This is also an expensive path (Table XII). It needs to go through the PMP, to the LSQ, then to the load unit, the cache where we get a cache hit and the data is sent back to the load queue. But it earns more IPC than it loses clock frequency. Note that even when the fall-through and the translation bypass are entirely disabled, the issue stage path is still around **23 ns** which means it is still a big bottleneck in terms of timing despite all the already implemented optimizations to reduce it, such as multiple RS to reduce their length, precomputed registers to evict the RAT from the critical path or even a top-2 tournament tree to avoid a cascade of two top-1 tournament trees. And reducing the size of the data structures inside it would only hurt performance.

The final result is that this configuration, despite showing a good gain in IPC, proves itself useful only on memory intensive applications such as MNIST or when the data is large enough to provoke cache misses. To improve the timing in a significant way we would probably need to modify heavily the current design. We could probably add a new stage by splitting the issue stage in half. One stage would only manage the allocation and the other the dispatch. This would cut this critical path almost in the middle (the tournament tree appears around 12.5 ns in the timing report). As for the critical path induced by the translation bypass in the LSQ, it could be shortened easily. Currently, we wait to resolve dependencies between loads and stores before dispatching to the load unit. We could ignore that verification. When a load is translated, we flag it as valid as soon as the physical address is received and no exception has occurred. And 1 cycle later we cancel it if a dependency has been found. We still perform forwarding the same cycle when receiving the physical address of a store or a load and give this value precedence over cache data. And if the PMP costs too much time we can do the same and wait only the next cycle to cancel the faulty load. This means we lose time only when a partial match occurs, which never happens across all benchmarks. And if necessary we could implement byte-to-byte store-to-load forwarding to resolve this issue.

### *D. Potential optimizations*

To give an idea of the theoretical limit of this design we ran a simulation with these parameters: ROB=32, Rollback=8, REG=70, LSQ_BYPASS=8, LSQ=8, RS=8. This configuration consumes more than 60,000 LUTs for the core only and we could not get accurate timing data since the implementation did not finish, but it is safe to assume that the critical path would exceed 30 ns. This version could not even get an average of +10% IPC (Table XIII). FFT gains the most with its store drain bottleneck greatly reduced (stores now wait for a token 22% of the time instead of 40%). MNIST shows good gains due to the larger windows, and CoreMark shows almost no gain, confirming it is heavily front-end bottlenecked (Table IV). This simulation shows that the current configuration finds a good balance.

But it also proves that this design needs real modifications to be improved and we will go over some possible optimizations. We could try to implement an early wakeup for loads which has a good upper bound gain (Table XIV). We implemented a prototype with the results here (Table XV). On FFT the potential counter dropped from 10.5% to 6.75% but the IPC gain stayed below 1% (1.278 $\rightarrow$ 1.286). On CoreMark the potential counter dropped from 20% to 6% but again with no significant performance gain. And on MNIST from 23.3% to 17%, again with no real gain. We could also extend the early wakeup from only fixed latency units to the multiplication.

Just as the early load wakeup it has an interesting potential. But again this is an upper bound estimation calculated by checking if an instruction needs the result 1 cycle after the write back. It is possible that this operand is not the only one missing for the said instruction. And as seen with the early load wakeup the results can be mixed.

Another interesting modification could be adding a second address-translation port (duplicated PMP and, when enabled, a second DTLB lookup port). This would remove the contention between loads and stores on the single translation port, which accounts for up to 12–14% of load waiting time on FFT and MNIST. The PMP itself is purely combinational and easy to duplicate (≈1–2k LUTs). The main cost lies in the LSQ, which would have to select and process two translations per cycle on an already critical path. Since the data cache still accepts a single load per cycle, we expect a modest IPC gain (a few percent) unless a second cache load port is added as well.

To conclude, at this state, almost all easy optimizations have been done. We have seen that increasing data structure sizes even to a very high degree provides a small return in IPC/LUT or in IPC gains at maximum clock frequency. We have seen that decreasing data structure sizes does not return enough clock frequency to justify the loss in IPC. The only way to really push this design forward would be to implement big modifications: dual issue MMU and PMP, dual issue cache, speculative loads with replay or even a run-ahead mode. And timing-wise, a new stage and a refactoring of the LSQ. Moreover, simulation data shows a front-end bottleneck (between 3% and 17%), indicating that a deep rework of the design would be necessary to improve the current design and modifications should go beyond the issue stage and execute stage.

### *E. Conclusion*

This article serves as an interesting experience: implementing an Out of Order design on a small RISC-V CPU to see the results and try to identify the potential gains and limits. The implementation is successful, the results are good and show the real gain of the OoO architecture even on a small CPU. However, the FPGA platform holds back its potential. The frequency penalties are heavier on FPGA, where routing dominates and associative structures map poorly to LUTs. Moreover, on an ASIC with a memory hierarchy several hundred cycles deep, the trade-off would shift towards OoO execution even more.

TABLE VII

CYCLES MEASURED ON THE FPGA (MEAN OF TWO CONSECUTIVE RUNS) AND IPC. COREMARK CYCLES COVER THE TIMED REGION (3 ITERATIONS, READ FROM MCYCLE); CM/MHZ IS THE RESULTING COREMARK/MHZ SCORE, GIVEN FOR COMPARISON ONLY (THE RUN IS SHORTER THAN THE 10 S REQUIRED BY THE EEMBC REPORTING RULES).

| Version | FFT | | MNIST | | CoreMark | | |
|---|---|---|---|---|---|---|---|
| | Cycles | IPC | Cycles | IPC | Cycles | IPC | CM/MHz |
| In-order 1-way SB16 | 143 794 | 0.770 | 1 711 346 | 0.654 | 1 178 576 | 0.699 | 2.55 |
| In-order 1-way SB8 | 143 794 | 0.770 | 1 711 346 | 0.654 | 1 178 576 | 0.699 | 2.55 |
| In-order 2-way SB16 | 116 770 | 0.948 | 1 280 893 | 0.874 | 951 689 | 0.866 | 3.15 |
| In-order 2-way SB8 | 116 553 | 0.949 | 1 282 828 | 0.873 | 952 434 | 0.865 | 3.15 |
| OoO 1-way ROB16 | 132 268 | 0.837 | 1 256 461 | 0.891 | 1 058 193 | 0.779 | 2.84 |
| OoO 1-way ROB8 | 151 644 | 0.730 | 1 524 059 | 0.735 | 1 124 523 | 0.733 | 2.67 |
| OoO 2-way ROB16 | 100 463 | 1.101 | 791 221 | 1.415 | 774 295 | 1.064 | 3.87 |
| OoO 2-way ROB8 | 130 170 | 0.850 | 1 303 070 | 0.859 | 918 928 | 0.897 | 3.26 |

TABLE VIII

BENEFIT OF OUT-OF-ORDER EXECUTION: EACH OOO VERSION COMPARED WITH THE IN-ORDER VERSION OF THE SAME ISSUE WIDTH, WITH A ROB AS LARGE AS THE IN-ORDER SCOREBOARD (CYCLE GAIN, LUT OVERHEAD, $F_{max}$ CHANGE, AND EXECUTION-TIME GAIN AT $F_{max}$).

| Issue width, size | FFT | MNIST | CoreMark | Geomean | LUTs | $F_{max}$ | Gain at $F_{max}$ |
|---|---|---|---|---|---|---|---|
| 1-way, SB16 / ROB16 | +8.7% | +36.2% | +11.4% | **+18.1%** | +51.9% | −10.2% | +6.1% |
| 1-way, SB8 / ROB8 | −5.2% | +12.3% | +4.8% | **+3.7%** | +49.5% | +0.1% | +3.9% |
| 2-way, SB16 / ROB16 | +16.2% | +61.9% | +22.9% | **+32.2%** | +45.2% | −0.8% | +31.2% |
| 2-way, SB8 / ROB8 | −10.5% | −1.6% | +3.6% | **−3.0%** | +37.2% | −21.2% | −23.6% |

TABLE XI

TIMING COST OF THE SAME-CYCLE PATHS OF THE OOO DUAL-ISSUE ROB16 VERSION: FALLTHROUGH OF THE RESERVATION STATIONS (RS FT) AND OF LSQ_BYPASS (BYPASS FT), AND SENDING A LOAD TO THE CACHE IN THE CYCLE OF ITS ADDRESS TRANSLATION (TRANSL.→CACHE). EACH LINE REMOVES ONE MORE PATH.

| Variant | Same-cycle path kept | | | $T_{min}$ (ns) | $F_{max}$ (MHz) | $F_{max}$ vs. base | Critical path |
|---|---|---|---|---|---|---|---|
| | RS FT | Bypass FT | Transl.→cache | | | | |
| Baseline | ✓ | ✓ | ✓ | 23.71 | 42.2 | — | RS entry → tournament → IRO |
| No RS fallthrough | — | ✓ | ✓ | 23.26 | 43.0 | +1.9% | RS entry → tournament → IRO |
| No fallthrough | — | — | ✓ | 23.02 | 43.4 | +3.0% | LSQ: translation → cache → LQ |
| No fallthrough, no translation bypass | — | — | — | 23.00 | 43.5 | +3.1% | RS entry → tournament → IRO |

TABLE IX

COMMIT IPC IN SIMULATION OF THE OOO DUAL-ISSUE ROB8 VERSION WITH 40 AND 44 PHYSICAL REGISTERS. WITH 40 REGISTERS, ONLY 40 − 32 − 2 = 6 CAN BE RENAMED AT ONCE (32 ARCHITECTURAL, 2 PRE-ALLOCATED), FEWER THAN THE 8 ROB ENTRIES, AND DECODE STALLS FOR LACK OF A FREE REGISTER 3 TO 19% OF THE TIME. WITH 44 REGISTERS THIS STALL DISAPPEARS.

| Benchmark | 40 phys. reg. | 44 phys. reg. | Diff. |
|---|---|---|---|
| FFT | 0.930 | 0.969 | +4.2% |
| MNIST | 0.873 | 0.934 | +7.0% |
| CoreMark | 0.965 | 0.981 | +1.7% |
| Geomean | — | — | **+4.3%** |

TABLE X

AREA OF THE OUT-OF-ORDER STRUCTURES IN THE OOO DUAL-ISSUE ROB16 CORE. THE LSQ REPLACES THE IN-ORDER LSU_BYPASS (297 LUTS IN SB16); THE ROB REPLACES THE SCOREBOARD. RESERVATION STATIONS INCLUDE THEIR TOURNAMENT TREES.

| OoO structure | LUTs | % of OoO core |
|---|---|---|
| RAT (speculative + commit) | 1 657 | 5.2% |
| Reservation stations (2) | 1 329 | 4.1% |
| `lsq_bypass` | 534 | 1.7% |
| Load/store queue | 7 330 | 22.9% |
| **New OoO structures** | **10 850** | **33.9%** |
| ROB (in-order SB16: 4 885) | 6 967 | 21.7% |
| **OoO structures + ROB** | **17 817** | **55.6%** |

TABLE XII

PERFORMANCE COST OF REMOVING EVERY SAME-CYCLE PATH (OOO DUAL ISSUE ROB16). POSITIVE DIFFERENCES MEAN MORE CYCLES OR A LONGER EXECUTION TIME. BASELINE: MEAN OF TWO RUNS; VARIANT: SINGLE RUN.

| | **Baseline** | **No FT, no bypass** | **Diff.** |
|---|---|---|---|
| FFT (cycles) | 100 463 | 114 701 | +14.2% |
| MNIST (cycles) | 791 221 | 971 865 | +22.8% |
| CoreMark (cycles) | 774 295 | 860 285 | +11.1% |
| Cycles, geomean | — | — | +15.9% |
| $F_{\max}$ (MHz) | 42.2 | 43.5 | +3.1% |
| **Execution time at $F_{\max}$** | — | — | **+12.5%** |

TABLE XIII

SCALING STUDY: OOO DUAL ISSUE WITH EVERY STRUCTURE DOUBLED (ROLLBACK 8) AGAINST ROB16. IPC FROM SIMULATION FOR BOTH. THE SYNTHESIS OF THE LARGE VERSION DID NOT COMPLETE: ITS LUTS AND $T_{\min}$ ARE LOWER BOUNDS, SO ITS GAINS ARE UPPER BOUNDS.

| | **OoO 2-way ROB16** | **OoO 2-way ROB32** | **Diff.** |
|---|---|---|---|
| *Parameters* | | | |
| Reg / RS / LSQ / Byp | 48 / 4 / 4 / 4 | 70 / 8 / 8 / 8 | — |
| *IPC (simulation)* | | | |
| FFT | 1.278 | 1.460 | +14.2% |
| MNIST | 1.453 | 1.595 | +9.8% |
| CoreMark | 1.162 | 1.175 | +1.1% |
| Geomean | — | — | **+8.2%** |
| *Hardware cost (large configuration: estimates)* | | | |
| Core LUTs | 33 794 | > 60 000 | ≥ +77.5% |
| $T_{\min}$ (ns) | 23.71 | > 30 | ≥ +26.5% |
| $F_{\max}$ (MHz) | 42.2 | < 33.3 | ≤ −21.0% |
| *Performance at $F_{\max}$ (upper bound)* | | | |
| FFT | — | — | ≤ −9.7% |
| MNIST | — | — | ≤ −13.2% |
| CoreMark | — | — | ≤ −20.1% |
| Geomean | — | — | **≤ −14.5%** |
| IPC / LUT | — | — | ≤ −39.0% |

TABLE XIV

UPPER BOUND OF THE GAIN OF OPTIMISATIONS EVALUATED BY INSTRUMENTATION, IN % OF CYCLES: INSTRUCTIONS ISSUED THE CYCLE AFTER THEIR PRODUCER'S WRITE-BACK (MUL OR LOAD).

| **OoO** | **Bench.** | **MUL wakeup** | **Load wakeup** |
|---|---|---|---|
| 1-way ROB16 | FFT | 1.0% | 6.4% |
| | MNIST | 8.6% | 12.4% |
| | CoreMark | 0.1% | 9.2% |
| 1-way ROB8 | FFT | 0.9% | 5.6% |
| | MNIST | 10.0% | 10.8% |
| | CoreMark | 0.2% | 8.7% |
| 2-way ROB16 | FFT | 3.1% | 10.5% |
| | MNIST | 28.2% | 23.3% |
| | CoreMark | 2.9% | 20.3% |
| 2-way ROB8 | FFT | 2.0% | 6.8% |
| | MNIST | 14.7% | 12.9% |
| | CoreMark | 1.7% | 14.7% |
| 2-way ROB32 | FFT | 4.0% | 12.7% |
| | MNIST | 30.7% | 31.2% |
| | CoreMark | 3.3% | 20.9% |

TABLE XV

EARLY WAKEUP OF LOAD CONSUMERS (WOKEN BY THE CACHE RESPONSE, ONE CYCLE BEFORE WRITE-BACK) ON THE OOO DUAL-ISSUE ROB16. PERF. AT $F_{\max}$ = IPC RATIO × $F_{\max}$ RATIO.

| | **ROB16** | **+ early wakeup** | **Diff.** |
|---|---|---|---|
| *Hardware cost* | | | |
| Core LUTs | 33 794 | 33 573 | −0.7% |
| $T_{\min}$ (ns) | 23.71 | 23.97 | +1.1% |
| $F_{\max}$ (MHz) | 42.2 | 41.7 | −1.1% |
| *IPC (simulation)* | | | |
| FFT | 1.278 | 1.286 | +0.6% |
| MNIST | 1.453 | 1.466 | +0.9% |
| CoreMark | 1.162 | 1.182 | +1.7% |
| Geomean | — | — | **+1.1%** |
| Perf. at $F_{\max}$ | — | — | **0.0%** |

# VI. APPENDIX: DETAILED RESULTS

## VII. References

**[1]** Official documentation of the CVA6 architecture by the OpenHW Group (https://docs.openhwfoundation.org/projects/cva6-user-manual/)

**[2]** Official GitHub repository of the CV32A6 softcore contest by Thales (https://github.com/ThalesGroup/cva6-softcore-contest)

**[3]** GitHub repository of the CV32A6 Out of Order (https://github.com/TelmoFieux/cva6-custom-reg-instr)